

```
[2] ▶ def dls(state, goal, path, visited, depth):  
  
    if state == goal:  
        return path  
  
    if depth == 0:  
        return None  
  
    blank = state.index(0)  
  
    moves = {  
        0: [1, 3],  
        1: [0, 2, 4],  
        2: [1, 5],  
        3: [0, 4, 6],  
        4: [1, 3, 5, 7],  
        5: [2, 4, 8],  
        6: [3, 7],  
        7: [4, 6, 8],  
        8: [5, 7]  
    }  
  
    for new_position in moves[blank]:  
  
        new_state = list(state)  
  
        new_state[blank], new_state[new_position] = \  
            new_state[new_position], new_state[blank]  
  
        new_state = tuple(new_state)  
  
        if new_state not in visited:  
  
            visited.add(new_state)
```

```
[2] if new_state not in visited:

    visited.add(new_state)

    result = dls(
        new_state,
        goal,
        path + [new_state],
        visited,
        depth - 1
    )

    if result is not None:
        return result

    visited.remove(new_state)

return None

print("Enter initial state:")
initial = tuple(map(int, input().split()))

print("Enter goal state:")
goal = tuple(map(int, input().split()))

for depth in range(31):

    visited = {initial}

    path = dls(
        initial,
        goal,
        [initial],
        visited,
        depth
    )

    if path is not None:
```

```
[2] depth
    )

    if path is not None:

        print("\nSolution found!")
        print("Depth:", depth)
        print("Path Cost:", len(path) - 1)

        print("\nPath:")

        for state in path:
            print(state)

        break

    else:
        print("No solution found.")

... Enter initial state:
1 2 3 0 4 6 7 5 8
Enter goal state:
1 2 3 4 5 6 7 8 0

Solution found!
Depth: 3
Path Cost: 3

Path:
(1, 2, 3, 0, 4, 6, 7, 5, 8)
(1, 2, 3, 4, 0, 6, 7, 5, 8)
(1, 2, 3, 4, 5, 6, 7, 0, 8)
(1, 2, 3, 4, 5, 6, 7, 8, 0)
```